

PART A : Prerequisites for Logistic Regression implementation

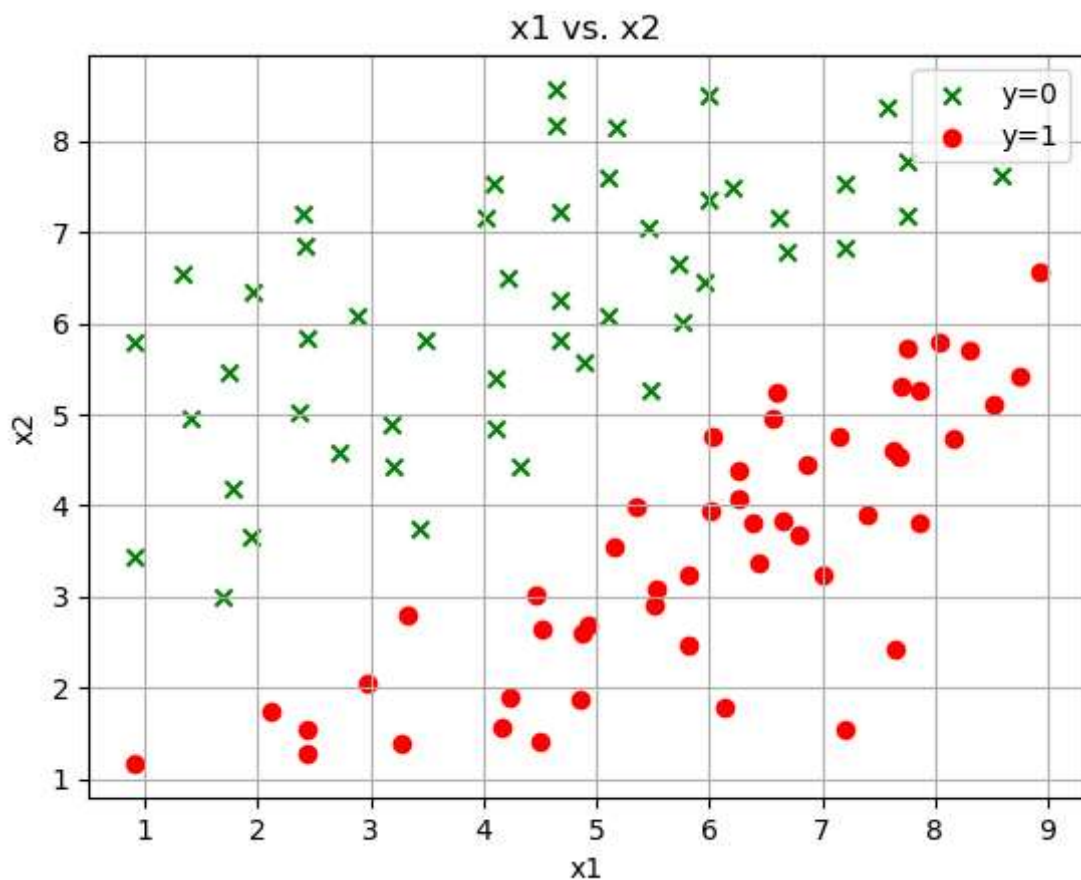
1. Plot the attached dataset data1.csv using scatter plot. There is a target feature ,Y, with discrete values 0,1. If the target feature is 1, the samples should be shown as red circle. If the target feature is 0, the samples should be shown as green x

```
In [1]: import pandas as pd
import matplotlib.pyplot as plt

dataset=pd.read_csv("data1.csv")
grouped=dataset.groupby('y')

for key,group in grouped:
    if key == 1:
        plt.scatter(group['x1'],group['x2'],c='red',marker='o',label='y=1')
    else:
        plt.scatter(group['x1'],group['x2'],c='green',marker='x',label='y=0')

plt.xlabel('x1')
plt.ylabel('x2')
plt.title('x1 vs. x2')
plt.grid(True)
plt.legend()
plt.show()
```



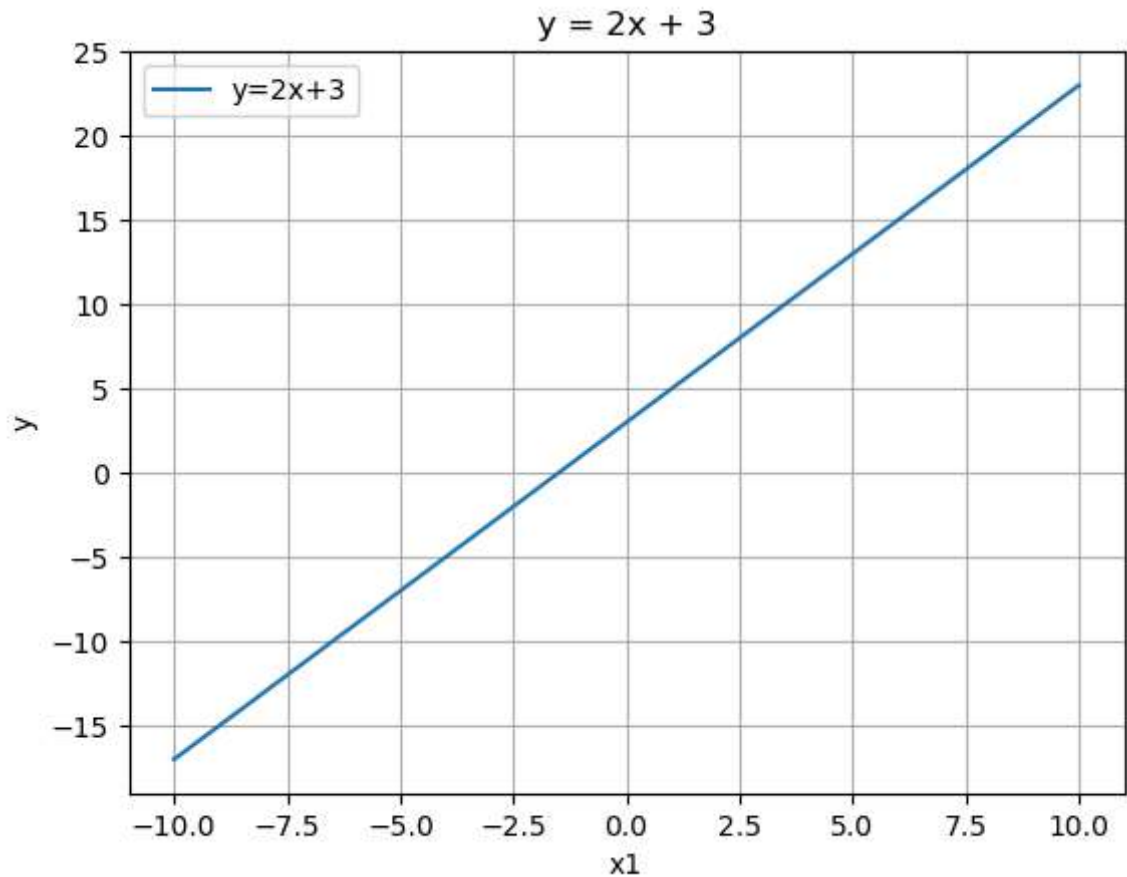
1. Plot a line $y=(2x+3)$

```
In [2]: import numpy as np
```

```

x_values=np.linspace(-10,10,50)
y_values=2*x_values + 3
plt.plot(x_values, y_values, label='y=2x+3') # plot with Label
plt.xlabel('x1')
plt.ylabel('y')
plt.title('y = 2x + 3')
plt.grid(True)
plt.legend()
plt.show()

```



In [3]: `dataset.head()`

Out[3]:

	x1	x2	y
0	4.5192	2.6487	1
1	2.4443	1.5438	1
2	4.2409	1.8990	1
3	5.8097	2.4711	1
4	6.4423	3.3590	1

1. Define a function `sigmoid(z)` that takes one parameter `z` and computes $1/(1+e^{-z})$. Create a vector `V` with 10 values randomly in the range `[-1000,1000]`. Transform `V` to `V'` that consists of respective sigmoid values using the defined function. Observe the range of output values in `V'`

In [4]:

```

def sigmoid(z):
    sig=1/(1+np.exp(-z))
    return sig

```

```
V=np.random.randint(-1000,1000,10)
V_prime=sigmoid(V)

# Observe the range of output values in V'
print("Minimum value in V':", np.min(V_prime))
print("Maximum value in V':", np.max(V_prime))
for v, v_prime in zip(V, V_prime):
    print(f"{v}\t{v_prime}")
```

```
Minimum value in V': 0.0
Maximum value in V': 1.0
-893    0.0
-431    6.592927495525641e-188
322     1.0
-56     4.780892883885469e-25
-602    3.5869216819018034e-262
301     1.0
-703    4.9088439016919216e-306
-655    3.444406129188316e-285
-317    2.131322828462168e-138
-474    1.3944905574373912e-206
```

```
C:\Users\garla\AppData\Local\Temp\ipykernel_24972\3039962399.py:2: RuntimeWarning:
overflow encountered in exp
  sig=1/(1+np.exp(-z))
```

1. Define a function hypothesis(theta, X) that takes two vectors as parameters, theta and X. If sigmoid(theta.X) >= 0.5, output 0 else output 1.

```
In [5]: def hypothesis(th, x):
        z = np.dot(x, th)
        sig = sigmoid(z)
        h = np.where(sig >= 0.5, 1, 0)
        return h
```

1. Define a function cost(theta,X,y) to compute the error

Error = $\frac{1}{m} \sum -y_i \log(h\theta(x_i)) - (1-y_i) \log(1-h\theta(x_i))$ Where x_i is the i th sample and y_i is the i th label, $h\theta(x_i)$ is the hypothesis(theta, x_i)

```
In [6]: def cost(theta, X, y):
        m = len(y)
        z = np.dot(X, theta)
        h = sigmoid(z)
        epsilon = 1e-10 # small value to avoid log(0)
        cost = (-1/m) * np.sum(y * np.log(h + epsilon) + (1 - y) * np.log(1 - h + epsilon))
        return cost
```

PART B : Implementation of logistic regressio

Implement gradient descent algorithm for logistic regression in data set loan_data.csv.

```
In [7]: import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
```

```

In [32]: # Read dataset
dataset = pd.read_csv("loan_data.csv").dropna()

In [33]: # Drop Loan_ID column
dataset = dataset.drop(columns=['Loan_ID'])

In [10]: # Convert categorical variables to dummy/indicator variables
dataset = pd.get_dummies(dataset, drop_first=True)

In [11]: # Split data into training and testing sets
train_set, test_set = train_test_split(dataset, test_size=0.2, random_state=42)

In [12]: # Separate features and labels+
X_train = train_set.drop('Loan_Status_Y', axis=1).values
Y_train = train_set['Loan_Status_Y'].values
X_test = test_set.drop('Loan_Status_Y', axis=1).values
Y_test = test_set['Loan_Status_Y'].values

In [13]: # Normalize feature matrix
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

In [14]: # Add column of ones for bias term
X_train = np.hstack((np.ones((X_train.shape[0], 1)), X_train))
X_test = np.hstack((np.ones((X_test.shape[0], 1)), X_test))

In [15]: theta = np.zeros(X_train.shape[1])

In [16]: # Sigmoid function
def sigmoid(z):
    z = np.clip(z, -500, 500) # prevent overflow
    return 1 / (1 + np.exp(-z))

In [17]: # Cost function
def cost(theta, X, y):
    m = len(y)
    z = np.dot(X, theta)
    h = sigmoid(z)
    error = (-1/m) * np.sum(y * np.log(h + 1e-10) + (1 - y) * np.log(1 - h + 1e-10))
    return error

In [18]: # Gradient descent function
def gradient_descent(X, y, theta, alpha=0.01, max_iters=1000, tol=1e-6, verbose=False):
    m = len(y)
    cost_history = []
    for i in range(max_iters):
        h = sigmoid(np.dot(X, theta))
        gradient = (1/m) * np.dot(X.T, (h - y))
        theta -= alpha * gradient
        cost_val = cost(theta, X, y)
        cost_history.append(cost_val)
        if verbose and i % 100 == 0:
            print(f"Iteration {i}: Cost = {cost_val:.6f}")
        if i > 0 and abs(cost_history[-2] - cost_history[-1]) < tol:
            break
    return theta, i + 1, cost_history

```

```
In [19]: # Run gradient descent
theta_est, iters, cost_hist = gradient_descent(X_train, Y_train, theta, alpha=0.001
```

```
In [20]: print("Estimated theta values:\n", theta_est)
print("Number of iterations:", iters)
```

Estimated theta values:
[nan nan nan nan nan nan nan nan nan nan nan nan nan nan nan]
Number of iterations: 1000

```
In [21]: # Predict on test data
y_pred_prob = sigmoid(np.dot(X_test, theta_est))
y_pred = (y_pred_prob >= 0.5).astype(int)
```

```
In [22]: accuracy = np.mean(y_pred == Y_test)
print("Accuracy on test set:", accuracy)
```

Accuracy on test set: 0.34959349593495936

PART C

```
In [23]: from sklearn.linear_model import LogisticRegression
from sklearn.metrics import confusion_matrix, accuracy_score, classification_report
```

```
In [24]: dataset = dataset.dropna()
```

```
In [25]: train_set, test_set = train_test_split(dataset, test_size=0.2, random_state=42)
X_train = train_set.drop('Loan_Status_Y', axis=1)
Y_train = train_set['Loan_Status_Y']
X_test = test_set.drop('Loan_Status_Y', axis=1)
Y_test = test_set['Loan_Status_Y']
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

```
In [26]: # 1. Use sklearn built in function to train the model with the trainset in part B
model = LogisticRegression()
model.fit(X_train, Y_train)
```

```
Out[26]: LogisticRegression
LogisticRegression()
```

```
In [27]: # 2. Compute confusion matrix
y_pred = model.predict(X_test)
cm = confusion_matrix(Y_test, y_pred)
print("Confusion Matrix:\n", cm)
```

Confusion Matrix:
[[17 16]
[1 72]]

```
In [28]: # 3. Compute the accuracy score.
accuracy = accuracy_score(Y_test, y_pred)
print("Accuracy:", accuracy)
```

Accuracy: 0.839622641509434

```
In [29]: # 4. Print a classification report using the following sklearn function
report = classification_report(Y_test, y_pred)
print("Classification Report:\n", report)
```

Classification Report:				
	precision	recall	f1-score	support
False	0.94	0.52	0.67	33
True	0.82	0.99	0.89	73
accuracy			0.84	106
macro avg	0.88	0.75	0.78	106
weighted avg	0.86	0.84	0.82	106

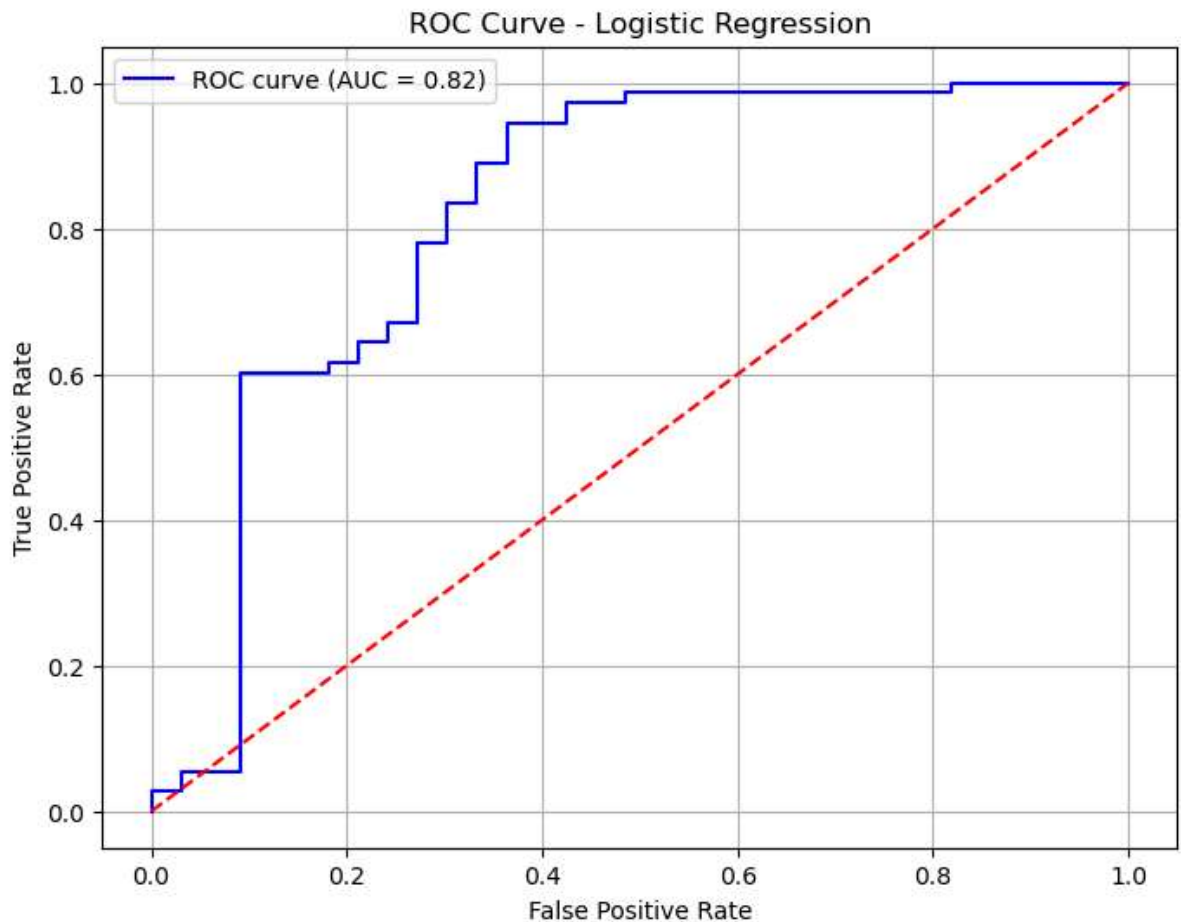
```
In [30]: # 5. Plot ROC curve for loan status
import matplotlib.pyplot as plt
from sklearn.metrics import roc_curve, roc_auc_score

# Get predicted probabilities for the positive class
y_prob = model.predict_proba(X_test)[: , 1]

# Compute ROC curve
fpr, tpr, thresholds = roc_curve(Y_test, y_prob)

# Compute AUC (Area Under Curve)
auc_score = roc_auc_score(Y_test, y_prob)

# Plot ROC curve
plt.figure(figsize=(8,6))
plt.plot(fpr, tpr, color='blue', label=f'ROC curve (AUC = {auc_score:.2f})')
plt.plot([0,1], [0,1], color='red', linestyle='--')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC Curve - Logistic Regression')
plt.legend()
plt.grid(True)
plt.show()
```



Custom Gradient Descent Logistic Regression:

- Implemented logistic regression using a custom gradient descent algorithm.
- Observed numerical instability in the training process, indicated by NaN values in the theta coefficients. This is likely due to overflow in the exponential function ($\text{np.exp}(-z)$) or division by zero in $\log(h)$ or $\log(1-h)$.
- The overall accuracy was low at 34%, indicating poor generalization and unstable optimization.

Scikit-learn Logistic Regression:

- Used `sklearn.linear_model.LogisticRegression` for model training.
- Achieved 83% accuracy, showing strong predictive performance.
- The confusion matrix indicates correct classification for both classes, with stable optimization and well-calibrated predictions.
- Overall, the built-in model is more robust, faster, and reliable compared to the custom gradient descent implementation.